

Reto 1. Guardar información en el localStorage

[Guardado de la lista de la compra en el localStorage](#)

[Mejorando la actualización por medio de un efecto](#)

[Recuperación de la lista de la compra del localStorage](#)

[Uso de useState con una función inicializadora](#)

[Integración en el componente `ShoppingList`](#)

Guardado de la lista de la compra en el localStorage

Ahora vamos a integrar una funcionalidad importante en nuestro componente `ShoppingList` : guardar la lista actual de elementos de la lista de la compra en el `localStorage` del navegador cada vez que se crea un nuevo elemento. Esta funcionalidad es esencial para mantener la persistencia de los datos, permitiendo que los elementos de la lista de la compra permanezcan disponibles incluso después de recargar la página.

Para lograr este objetivo, realizaremos una pequeña modificación en la función `handleSubmit` dentro del componente `ShoppingList` . Cada vez que se añada un nuevo elemento, guardaremos la lista actualizada de elementos de la lista de la compra en `localStorage` .

Aquí está el código actualizado del componente `ShoppingList` :

```
import { useState } from 'react';
import ShoppingListForm from './ShoppingListForm';
import ShoppingListItems from './ShoppingListItems';

function ShoppingList() {
  const [shoppingItems, setShoppingItems] = useState([]);

  function handleSubmit(newItem) {
    setShoppingItems(prevItems => {
      const updatedItems = [...prevItems, newItem];
      // Guardar los elementos de la lista actualizados en localStorage
      localStorage.setItem('shoppingItems', JSON.stringify(updatedItems));
      return updatedItems;
    });
  }

  function handleDelete(itemIndex) {
    setShoppingItems(prevItems => {
      const updatedItems = prevItems.filter(
        (_, index) => index !== itemIndex
      );
      localStorage.setItem('shoppingItems', JSON.stringify(updatedItems));
      return updatedItems;
    });
  }

  return (
    <div>
      <ShoppingListForm onSubmit={handleSubmit} />
      <ShoppingListItems shoppingItems={shoppingItems} onDelete={handleDelete} />
    </div>
  );
}

export default ShoppingList;
```

En este código, hemos actualizado la función `handleSubmit` para que, después de añadir el nuevo elemento a `shoppingItems`, guarde automáticamente esta lista actualizada en `localStorage`. Utilizamos `localStorage.setItem` para guardar los datos, y como `localStorage` solo puede almacenar strings, convertimos nuestra lista de elementos de la lista de compra a un string JSON con `JSON.stringify`.

De manera similar, actualizamos la función `handleDelete` para asegurarnos de que la lista de elementos de la lista de la compra en `localStorage` se actualice también cuando eliminemos un elemento.

Con esta implementación, cada vez que el usuario añada o elimine un elemento, la lista actualizada de elementos de la lista de compra se guardará en `localStorage`. Esto asegura que la información de los elementos de la lista de compra se mantenga persistente en el navegador del usuario, proporcionando una experiencia de usuario más consistente y fiable. En el siguiente paso, veremos cómo recuperar estos elementos guardados del `localStorage` cuando el usuario vuelva a cargar la aplicación.

Mejorando la actualización por medio de un efecto

```
import { useState, useEffect } from 'react';
import ShoppingListForm from './ShoppingListForm';
import ShoppingListItems from './ShoppingListItems';

function ShoppingList() {
  const [shoppingItems, setShoppingItems] = useState([]);

  function handleSubmit(newItem) {
    setShoppingItems(prevItems => [...prevItems, newItem]);
  }

  function handleDelete(itemIndex) {
    setShoppingItems(prevItems => prevItems.filter((_, index) => index !== itemIndex));
  }

  useEffect(
    function () {
      window.localStorage.setItem('myShoppingList', JSON.stringify(shoppingItems));
    },
    [shoppingItems]
  );

  return (
    <div>
      <ShoppingListForm onSubmit={handleSubmit} />
      <ShoppingListItems shoppingItems={shoppingItems} onDelete={handleDelete} />
    </div>
  );
}

export default ShoppingList;
```

En esta versión del código, hemos mejorado la gestión del `localStorage` al utilizar el hook `useEffect`. Esta mejora aumenta la eficiencia y la legibilidad del código, además de garantizar que el `localStorage` refleje siempre el estado actual de la lista de compras.

En la función `handleSubmit`, en lugar de ocuparnos directamente de guardar el nuevo elemento en el `localStorage`, simplemente añadimos el nuevo elemento a la lista existente utilizando el estado `items` y `setItems`. De manera similar, en la función `handleDelete`, eliminamos un elemento de la lista utilizando el índice proporcionado.

La parte clave de este cambio es la incorporación del hook `useEffect`. Este hook se activará cada vez que `items` cambie, lo que sucede cuando se agrega un nuevo elemento o se elimina uno existente. Dentro del efecto, utilizamos `window.localStorage.setItem` para guardar la lista actualizada en el `localStorage`. Estamos almacenando la lista de elementos bajo la clave 'myShoppingList', y para ello, transformamos la lista en un formato JSON con `JSON.stringify(items)`.

La ventaja principal de esta implementación es que el `localStorage` se actualiza de forma automática en respuesta a cualquier cambio en la lista de compras. Esto evita la repetición de código para guardar en diferentes partes del componente y asegura que los datos almacenados en el `localStorage` siempre reflejen con precisión el estado actual de la lista de compras. Esto mejora la legibilidad y la eficiencia del código, y garantiza que la persistencia de datos en el navegador sea consistente y confiable.

Recuperación de la lista de la compra del localStorage

Para mejorar la experiencia del usuario en nuestra aplicación de la lista de la compra, es esencial recuperar y mostrar los elementos de la lista de la compra guardados en el `localStorage` cuando el usuario vuelve a cargar la página. En este paso, explicaremos cómo lograrlo de manera efectiva en nuestro componente `ShoppingList`.

Uso de useState con una función inicializadora

La clave para recuperar los elementos de la lista de compra almacenados en `localStorage` de forma óptima está en cómo inicializamos el estado de `shoppingItems` en el componente `ShoppingList`. Usaremos una función inicializadora con `useState`, como se muestra a continuación:

```
const [shoppingItems, setShoppingItems] = useState(function() {
  const storedItems = localStorage.getItem('shoppingItems');
  return storedItems === null ? [] : JSON.parse(storedItems);
});
```

En este fragmento, la función pasada a `useState` se ejecuta solo en el momento de montar el componente. Esto es particularmente útil por varias razones:

1. **Eficiencia en la ejecución:** Al usar una función inicializadora, el código que recupera los elementos de la lista de compra de `localStorage` se ejecuta solo una vez, cuando el componente se monta por primera vez. Esto es más eficiente que tener un código de inicialización que se ejecuta en cada renderizado.
2. **Verificación de datos nulos:** Antes de intentar analizar los datos con `JSON.parse`, comprobamos si `localStorage.getItem('shoppingItems')` devuelve `null`. Esto es crucial porque `JSON.parse(null)` resultaría en un error. Por lo tanto, si no hay elementos de la lista de compra almacenados (es decir, `storedItems` es `null`), devolvemos un arreglo vacío `[]`.
3. **Conversión segura de datos:** Si `localStorage` tiene elementos de la lista de compra almacenados (`storedItems` no es `null`), los convertimos de una cadena JSON a un objeto JavaScript utilizando `JSON.parse`. Esto reconstruye nuestro arreglo de elementos de la lista de compra tal y

como estaba antes de ser almacenado en `localStorage`.

Integración en el componente `ShoppingList`

Ahora, integraremos este enfoque en nuestro componente `ShoppingList`:

```
import { useState } from 'react';
import ShoppingListForm from './ShoppingListForm';
import ShoppingListItems from './ShoppingListItems';

function ShoppingList() {
  const [shoppingItems, setShoppingItems] = useState(function() {
    const storedItems = localStorage.getItem('shoppingItems');
    return storedItems === null ? [] : JSON.parse(storedItems);
  });

  // Resto del componente...
}

export default ShoppingList;
```

Con esta implementación, aseguramos que el estado `shoppingItems` se inicialice con los elementos de la lista de compra almacenados en `localStorage` si existen, o con un arreglo vacío si no hay elementos de la lista de compra almacenados. Este

enfoque optimiza el rendimiento al reducir las operaciones innecesarias y garantiza un manejo seguro de los datos almacenados, mejorando así la robustez de nuestra aplicación de la lista de la compra en React.